

Semana 11: Laboratório de Vulnerabilidades de Segurança de Memória

Segurança de Sistemas Informáticos, 2025-26

Licenciatura em Engenharia Informática, Universidade do Minho

Vítor Francisco Fonte

2026-04-18

Neste trabalho laboratorial, vai investigar, explorar, observar mitigações e remediar, de forma segura, duas classes de vulnerabilidades de corrupção de memória em C: buffer overflows de stack (Parte A) e vulnerabilidades de string de formato (Parte B). Todos os exercícios requerem um ambiente Linux x86-64 com GCC, GDB e Python 3; o *Guia de Configuração do Ambiente* enumera os caminhos de configuração suportados (Ubuntu nativo, WSL2, VirtualBox, Docker Desktop e UTM). Os alunos submetem um relatório em PDF através da plataforma de e-learning.

Prepare o seu ambiente antes da sessão de laboratório. A instalação única, em especial no caminho UTM recomendado para alunos com Macs Apple Silicon, pode demorar **60 a 90 minutos ou mais**: o UTM emula x86-64 em software e o Ubuntu 22.04 aplica vários anos de atualizações de segurança acumuladas durante a instalação. Este tempo pode ser superior ao da própria sessão de laboratório. Se chegar à aula sem um ambiente funcional, junte-se a um(a) colega cujo ambiente já esteja operacional, faça os exercícios da Semana 11 na máquina dele(a) e termine a sua própria instalação entre sessões.

Objetivos de Aprendizagem

Ao completar este trabalho, os alunos serão capazes de:

1. Identificar e explorar um buffer overflow de stack num programa em C.
 2. Identificar e demonstrar uma vulnerabilidade de string de formato num programa em C.
 3. Observar o efeito de mitigações do compilador e do sistema operativo sobre vulnerabilidades de corrupção de memória.
 4. Aplicar práticas de código seguro para remediar vulnerabilidades de buffer overflow e de string de formato.
-

Configuração

Este trabalho requer uma **shell Linux x86-64**. As *flags* de compilação e o layout da stack usados na Parte A e na Parte B são específicos desta arquitetura, e os resultados noutras plataformas (em particular em ARM64) serão diferentes.

Se ainda não configurou um ambiente desse tipo, o ponto de partida mais rápido é o **Assistente de Configuração** (abra `Lab - Assistente de Configuracao.html` dos materiais da disciplina em qualquer *browser*): este deteta automaticamente o seu sistema operativo e a família do CPU e mostra os comandos exatos para o seu caminho. Para a referência completa, incluindo resolução de problemas e os percursos detalhados dos caminhos baseados em VM, consulte o **Guia de Configuração do Ambiente** (documento separado). Esse guia cobre todos os caminhos suportados (Ubuntu nativo, WSL2, Mac Intel com VirtualBox, Mac Apple Silicon com Docker Desktop e Rosetta, e Mac Apple Silicon com UTM) e explica porque a escolha depende da família do seu CPU.

Com uma *shell* x86-64 em funcionamento, instale o *toolchain* da Semana 11:

```
# Dentro de uma shell de container Docker (já é root)
apt update && apt install -y gcc gdb python3

# Numa instalação nativa / VirtualBox / WSL2 / VM UTM
sudo apt update && sudo apt install gcc gdb python3
```

Crie uma diretoria de trabalho para este trabalho (todos os caminhos):

```
mkdir -p ~/ssi-lab-swsec && cd ~/ssi-lab-swsec
```

Obter os ficheiros iniciais

Os dois ficheiros fonte para este trabalho — `vuln.c` e `fmtvuln.c` — são distribuídos como um arquivo zip (`week11-files.zip`) juntamente com este enunciado na plataforma de e-learning da disciplina. Descarregue-o antes da sessão de laboratório e siga as instruções para o seu caminho abaixo.

Ubuntu nativo e WSL2: o zip já está na sua máquina Linux. Descomprima-o diretamente:

```
unzip week11-files.zip -d ~/ssi-lab-swsec
cd ~/ssi-lab-swsec
```

VirtualBox (Caminho 3) e UTM (Caminho 5): o zip está no seu *host* (Mac ou Windows), ainda não dentro da VM. Copie-o para a VM com SCP e depois descomprima dentro da VM. Execute o seguinte a partir do terminal do *host* (não da *shell* da VM):

```
# Se configurou o alias SSH durante a configuração do ambiente:
scp week11-files.zip ssi-lab:~
```

```
# Caso contrário (substitua <username> pelo nome de utilizador que
escolheu durante a instalação do Ubuntu):
scp -P 2222 week11-files.zip <username>@localhost:~
```

Depois, dentro da *shell* da VM:

```
unzip week11-files.zip -d ~/ssi-lab-swsec
cd ~/ssi-lab-swsec
```

Docker (Caminhos 3b e 4): descomprima no seu Mac — não dentro do *container* — na pasta montada via *bind-mount*:

```
unzip week11-files.zip -d ~/ssi-lab/work
```

Os ficheiros aparecem imediatamente em `/root/work` dentro da *shell* do *container*. Não é necessário copiar nada manualmente para dentro do *container*. Se o `unzip` não estiver disponível no seu Mac, instale-o com `brew install unzip` ou abra o zip no Finder.

Notas por plataforma para este trabalho

Alunos em Apple Silicon devem usar o Caminho 5 (UTM). O Exercício 2 abaixo foi construído em torno de uma sessão interativa de GDB (`break`, `run`, `info frame`, `print`). O Caminho 5 executa um *kernel* Linux x86-64 totalmente emulado sob UTM e suporta esse fluxo exatamente como descrito. O Caminho 4, mais leve (Docker com Rosetta), *não suporta*: o Rosetta não consegue fazer `ptrace` aos processos que traduz, e a combinação Apple Silicon com Rosetta suprime também a aleatorização PIE/ASLR do código do programa. Se ainda assim escolher o Caminho 4, tenha em conta as duas consequências abaixo e siga a *alternativa de análise estática* indicada no Exercício 2.

Consequências do Caminho 4 (apenas Apple Silicon com Docker e Rosetta):

1. **O GDB interativo (breakpoints, `info frame`, stepping) irá falhar** dentro do *container* com `Couldn't get registers: Input/output error`. O Exercício 2 abaixo fornece uma alternativa de análise estática que produz as mesmas respostas.
2. **PIE/ASLR do código do programa fica suprimida.** A cache de tradução do Rosetta fixa o código traduzido num endereço base estável, pelo que o endereço impresso de `secret_function` aparecerá idêntico em execuções sucessivas, mesmo com ASLR ativo. O endereço da stack varia. O Exercício 4 abaixo pede-lhe para relatar o que observa e descrever o comportamento esperado num x86-64 nativo.

Todos os outros caminhos (Ubuntu nativo, WSL2, Mac Intel com VirtualBox, Mac Intel com Docker Desktop, Apple Silicon com UTM) não têm ressalvas para este trabalho; o fluxo interativo de GDB no Exercício 2 funciona conforme descrito.

Parte A: Buffer Overflow de Stack (CWE-120)

O Programa Vulnerável

Crie um ficheiro chamado `vuln.c` com o seguinte conteúdo:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void secret_function(void) {
    printf("\n[!] ACCESS GRANTED: you reached the secret function!\n");
    printf("[!] In a real exploit, this could be arbitrary code
execution.\n\n");
    exit(0);
}

void process_input(char *input) {
    char buffer[64];
    printf("[*] Buffer is at:          %p\n", (void *)buffer);
    printf("[*] secret_function is at: %p\n", (void *)secret_function);
    strcpy(buffer, input); /* CWE-120: no bounds check */
    printf("[*] You entered: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <input>\n", argv[0]);
        return 1;
    }
    printf("[*] process_input return address is on the stack.\n");
    process_input(argv[1]);
    printf("[*] Normal programme termination.\n");
    return 0;
}
```

Exercícios

Exercício 1: Compilação sem mitigações

Compile o programa com todas as principais mitigações desativadas:

```
gcc -o vuln vuln.c -fno-stack-protector -z execstack -no-pie -g
```

Explique, no seu relatório, o que cada *flag* faz:

- `-fno-stack-protector`
- `-z execstack`

- `-no-pie`
- `-g`

Exercício 2: Execução normal e exploração com GDB

Execute o programa com uma entrada curta e benigna:

```
./vuln "Hello"
```

Agora inicie o GDB e examine o layout da stack:

```
gdb -q ./vuln
```

Dentro do GDB, defina um *breakpoint* em `process_input`, execute o programa com um argumento curto, e responda às seguintes questões no seu relatório:

1. Qual é o endereço de `buffer` na stack?
2. Qual é o endereço do *return address* guardado (use `info frame`)?
3. Quantos bytes separam o início de `buffer` do *return address* guardado?
4. Qual é o endereço de `secret_function`?

Uma sessão interativa típica de GDB tem esta forma:

```
(gdb) break process_input
(gdb) run Hello
(gdb) print &buffer                # responde à questão 1
(gdb) info frame                  # responde à questão 2 (procure
"saved rip at ...")
(gdb) print (void*)secret_function # responde à questão 4
```

A questão 3 é simplesmente a diferença aritmética entre os dois endereços.

Alternativa para Apple Silicon: método de desmontagem estática

Se estiver num Mac M-series e a execução de GDB produzir `Couldn't get registers: Input/output error`, responda às mesmas quatro questões sem se ligar a um processo vivo. Os quatro valores podem ser derivados apenas a partir (a) do próprio *output* `printf` do programa e (b) do assembly do binário compilado.

1. **Endereço de `buffer`**. O programa imprime-o em cada execução:

```
./vuln "Hello"
```

Procure a linha `[*] Buffer is at: 0x...`. Copie esse endereço: é a resposta à questão 1.

2. **Endereço de `secret_function`**. O programa também o imprime, na linha `[*] secret_function is at: 0x...`. Por o binário ter sido *linked* com `-no-pie`, este endereço é fixo entre execuções. Isto responde à questão 4.
3. **Offset de `buffer` até ao `return address` guardado**. O modo *batch* do GDB consegue ler o assembly do binário sem iniciar um processo (logo, não envolve o Rosetta):

```
gdb -q -batch -ex "disas process_input" ./vuln
```

Verá um *output* semelhante a:

```
Dump of assembler code for function process_input:
...
<+8>:  sub    $0x50, %rsp
...
<+16>:  lea     -0x40(%rbp), %rax    ; endereço de buffer
...
<+84>:  call    0x401080 <strcpy@plt>
...
```

Leia os *offsets* diretamente da desmontagem:

- A linha `lea -0x40(%rbp), %rax` imediatamente antes da chamada a `strcpy@plt` mostra que **buffer começa em `rbp - 0x40`** (64 bytes abaixo do RBP guardado).
- Em x86-64, o RBP guardado reside em `[rbp]` (8 bytes), imediatamente seguido pelo *return address* guardado em `[rbp + 8]`.
- Portanto, a distância em memória do primeiro byte de **buffer até ao *return address* guardado** é:

```
offset = 0x40    (buffer até saved RBP)
        + 0x08    (saved RBP até saved return address)
        = 0x48    = 72 bytes
```

Esta é a resposta à questão 3. Para Ubuntu 22.04 com GCC 11 a compilar o `vuln.c` fornecido, o **offset** é **72 bytes**. Versões diferentes do compilador podem produzir um *frame* ligeiramente diferente; leia sempre a partir da sua própria desmontagem.

4. **Endereço do *return address* guardado** (questão 2) é simplesmente `endereço_de_buffer + 72`.

No seu relatório, mostre o *output* da desmontagem, a aritmética e as quatro respostas. Explique brevemente (uma ou duas frases) por que razão a desmontagem estática é suficiente aqui: a *flag* `-no-pie` fixa os endereços de código do programa, o tamanho do *stack frame* é determinado em tempo de compilação, e o endereço de *buffer* é divulgado pelo próprio programa.

Exercício 3: Desencadear o overflow

Usando a informação do Exercício 2, construa uma entrada que sobrescreva o *return address* com o endereço de `secret_function`. Pode usar Python para gerar o *payload*:

```
./vuln $(python3 -c "import sys; sys.stdout.buffer.write(b'A'*<OFFSET> + b'<ADDRESS>')")
```

Substitua `<OFFSET>` pelo número de bytes de *padding* e `<ADDRESS>` pela representação *little-endian* do endereço de `secret_function`.

No seu relatório, documente: o *offset* exato, o endereço usado, o comando completo, e uma captura de ecrã do *output* a mostrar `ACCESS GRANTED`.

Exercício 4: Efeito das mitigações

Recompile o programa com mitigações ativas, uma de cada vez, e repita o ataque. Documente o resultado de cada caso:

Flags de compilação	Comportamento esperado
<code>gcc -o vuln vuln.c -z execstack -no-pie -g</code>	Stack canary ativo
<code>gcc -o vuln vuln.c -fno-stack-protector -z execstack -g</code>	PIE/ASLR ativo
<code>gcc -o vuln vuln.c -g</code>	Todas as mitigações por defeito

Para cada caso, responda: o *exploit* ainda funciona? Se não, que erro ou mensagem observa? Explique *por que razão* cada mitigação impede (ou não impede) o ataque.

Nota para alunos em Apple Silicon. No caso de PIE/ASLR, poderá observar que o endereço impresso de `secret_function` é idêntico entre execuções sucessivas, apesar de a ASLR estar nominalmente ativa dentro do *container*. Este é um efeito secundário da cache de tradução do Rosetta, que fixa o código traduzido num endereço base estável. Num ambiente Linux x86-64 nativo, o endereço do código variaria entre execuções. Relate o que observa e explique o que deveria acontecer em x86-64 nativo: descreva a aleatorização do lado da stack (o endereço `Buffer is at:` *efetivamente* muda entre execuções) e a aleatorização do lado do código (que mudaria em Linux nativo mas está aqui suprimida).

Exercício 5: Remediação segura

Modifique `vuln.c` para eliminar a vulnerabilidade de *buffer overflow*. Deve:

1. Substituir `strcpy` por uma alternativa segura.
2. Adicionar uma verificação explícita de comprimento.
3. Compilar com todas as mitigações por defeito ativas.

Inclua o código corrigido e uma breve justificação no seu relatório.

Parte B: Vulnerabilidade de String de Formato (CWE-134)

O Programa Vulnerável

Uma vulnerabilidade de string de formato surge quando uma entrada controlada pelo utilizador é passada diretamente como argumento de string de formato a uma função da família `printf`: por exemplo, escrever `printf(input)` em vez de `printf("%s", input)`. Quando o `printf` processa uma string de formato, lê um argumento da stack da chamada (ou de registos, em x86-64) para cada especificador de formato que encontra. Se quem chama não fornecer tais argumentos mas a string de formato contiver especificadores, o `printf` lerá silenciosamente quaisquer valores que por acaso estejam na stack.

Crie um ficheiro chamado `fmtvuln.c` com o seguinte conteúdo:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void process_input(char *input) {
    unsigned long secret = 0xcafebabe; /* a sensitive value resident on
the stack */

    printf("[*] Address of secret on stack: %p\n", (void *)&secret);
    printf("[*] Processing input...\n");
    printf(input); /* CWE-134: user input used as format
string */
    printf("\n");
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <input>\n", argv[0]);
        return 1;
    }
    process_input(argv[1]);
    printf("[*] Normal programme termination.\n");
    return 0;
}
```

A vulnerabilidade está na linha marcada com `CWE-134`. O programa também imprime deliberadamente o endereço da variável local `secret` para apoiar a sua investigação no Exercício 8.

Exercícios

Exercício 6: Compilação e diagnósticos do compilador

Compile o programa com símbolos de *debug* e com as *flags* de aviso sobre *format strings* do GCC ativas:

```
gcc -o fmtvuln fmtvuln.c -g -Wall -Wformat -Wformat-security
```

1. Registe os avisos produzidos pelo compilador. O que indica a mensagem de aviso sobre a vulnerabilidade?
2. Agora compile sem as *flags* de aviso e execute o programa com uma entrada normal e benigna:

```
gcc -o fmtvuln fmtvuln.c -g  
./fmtvuln "Hello, world!"
```

Confirme que o programa se comporta como esperado. No seu relatório, explique por que razão os avisos do compilador constituem uma forma útil, embora não suficiente, de análise estática para detetar vulnerabilidades de string de formato.

Exercício 7: Desencadear a vulnerabilidade

Execute o programa com uma string de formato como entrada:

```
./fmtvuln "%p %p %p %p %p %p %p %p %p %p"
```

1. O que observa no *output*? Em que difere da execução normal no Exercício 6?
2. Cada especificador `%p` faz com que o `printf` leia e imprima um valor com a largura de uma palavra a partir da stack da chamada. Num parágrafo curto, explique o mecanismo pelo qual isto ocorre: por que razão o `printf` não deteta que não foram passados argumentos correspondentes?
3. Repita a experiência usando `%x` no lugar de `%p`. Que diferença, se alguma, observa no *output*? Qual dos especificadores é mais apropriado para um processo de 64 bits, e porquê?

Exercício 8: Localizar um valor na stack

O programa imprime o endereço da variável local `secret` antes de processar a sua entrada. Usando esse endereço como referência, realize a seguinte investigação.

1. Execute o programa com uma sequência longa de especificadores `%p`. Pode gerar a entrada com Python:

```
./fmtvuln "$(python3 -c "print('%p ' * 30, end='')")"
```

2. Examine o *output*. Identifique o valor impresso que corresponde ao endereço de `secret`. Em seguida, procure o valor sentinela `0xcafebabe` no *output*. Note que o especificador `%p` do `printf` suprime zeros à esquerda, pelo que o valor aparecerá como `0xcafebabe` em vez de `0x00000000cafebabe`. Em que posição da sequência `%p` aparece?
3. No seu relatório, explique o que este exercício demonstra sobre o risco das vulnerabilidades de string de formato: que categoria de informação pode um atacante recuperar, e de que zona do espaço de endereçamento do processo?

Nota: O layout exato da stack pode variar entre sistemas e versões do compilador. Se o valor sentinela não aparecer dentro de 30 especificadores, aumente a contagem incrementalmente e repita.

Exercício 9: Remediação segura

Modifique `fmtvuln.c` para eliminar a vulnerabilidade de string de formato. A correção é uma alteração numa única linha: a chamada a `printf` deve tratar a entrada do utilizador como dados e não como string de formato.

1. Mostre a linha de código corrigida e explique por que razão é segura mesmo quando a entrada contém especificadores de formato como `%p` ou `%x`.
2. Recompile e verifique que a entrada do Exercício 7 já não divulga valores da stack.
3. Recompile com `-Wall -Wformat -Wformat-security` e confirme que não é produzido nenhum aviso.

Referências

- CWE-120: Buffer Copy without Checking Size of Input, <https://cwe.mitre.org/data/definitions/120.html>
- CWE-134: Use of Externally-Controlled Format String, <https://cwe.mitre.org/data/definitions/134.html>
- CWE-190: Integer Overflow or Wraparound, <https://cwe.mitre.org/data/definitions/190.html>
- OWASP Top 10 (2025), <https://owasp.org/Top10/>
- GDB Quick Reference, <https://sourceware.org/gdb/current/onlinedocs/gdb/>

Entrega

Façam o upload dos vossos entregáveis para o repositório privado do vosso grupo em <https://github.com/uminho-lei-ssi/> , seguindo o mesmo procedimento dos trabalhos anteriores. As respostas escritas devem ser submetidas como ficheiro Markdown; o código-fonte no seu formato original.